

Asymptotic analysis of Merge Sort

It involves understanding its time complexity, which is consistently $O(n \log n)$ in the worst, average, and best cases. Let's break down the analysis step by step.

Time Complexity Analysis:

- **Divide:** Dividing the array of size n takes $O(1)$ time.
- **Conquer:** The recursive calls on subproblems occur until each sublist contains only one element, resulting in $O(\log n)$ levels of recursion.
- **Combine (Merge):** Merging two sorted sublists of size $n/2$ takes $O(n)$ time.

Overall Time Complexity of Merge Sort can be expressed using following **recurrence relation** (discussed in the later)

$$T(n) = 2T(n/2) + O(n).$$

Binary Search

- Binary search. Given value and sorted array $a[]$, find index i such that $a[i] = \text{value}$, or report that no such index exists.

- Invariant. Algorithm maintains $a[\text{lo}] \leq \text{value} \leq a[\text{hi}]$.
- 
- | Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|-------|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| Value | 6 | 13 | 14 | 25 | 33 | 43 | 51 | 53 | 64 | 72 | 84 | 93 | 95 | 96 | 97 |

- Ex. Binary search for 33.

Binary Search

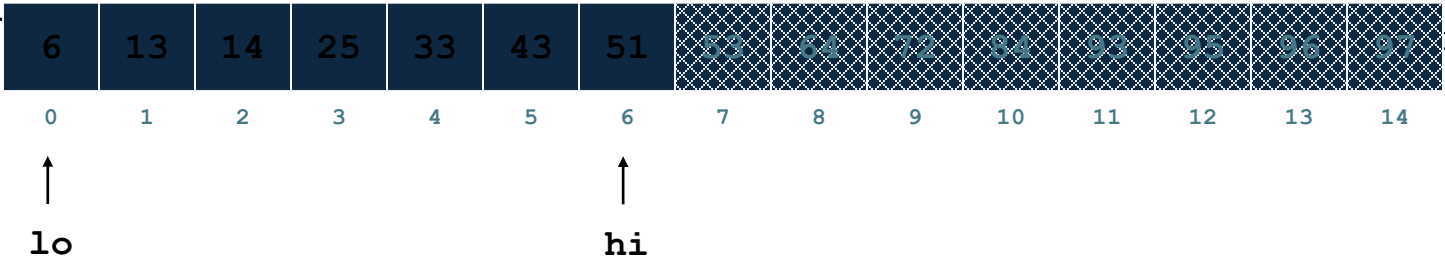
- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

- Invariant: $a[lo] \leq a[hi]$.

- Ex. Binary search for 33.

Binary Search

- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

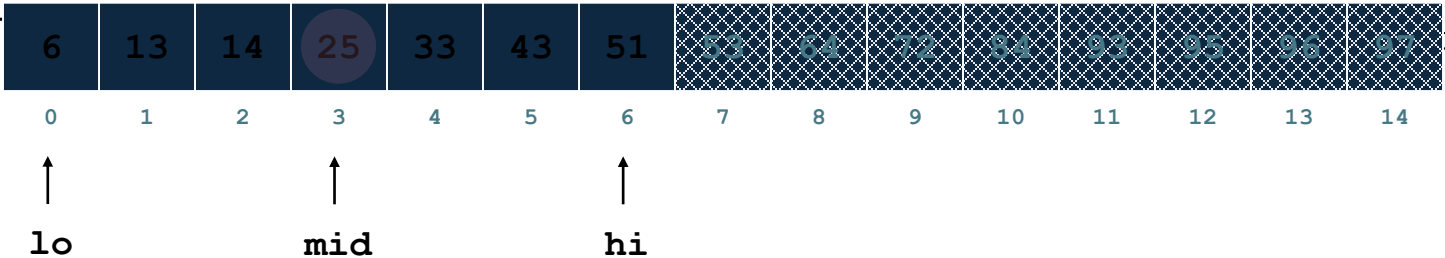
- Invariant `a[lo] ≤ value ≤ a[hi]`.


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
6	13	14	25	33	43	51	53	54	72	84	93	95	96	97

- Ex. Binary search for 33.

Binary Search

- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

- Invariant `lo ≤ value ≤ a[hi]`.


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
6	13	14	25	33	43	51	53	54	72	84	93	95	96	97

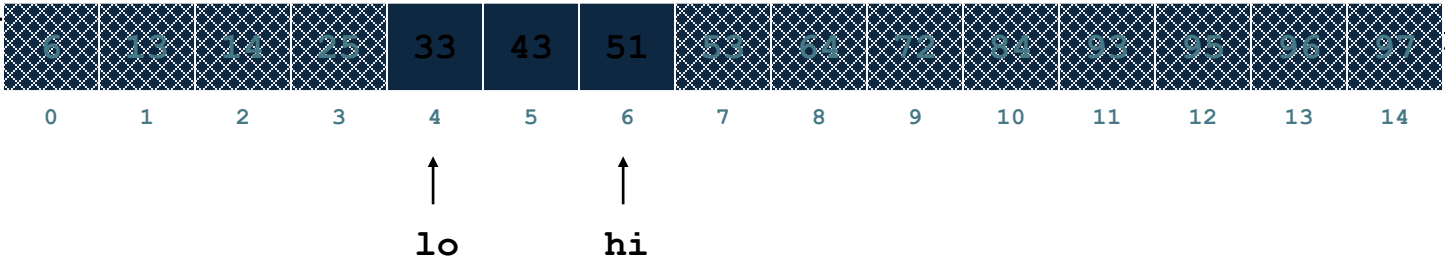
↑ ↑ ↑
`lo` `mid` `hi`

- Ex. Binary search for 33.

Binary Search

- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

- Invariant: `lo ≤ value ≤ a[hi]`.



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
6	13	14	35	33	43	51	53	54	72	84	93	95	96	97

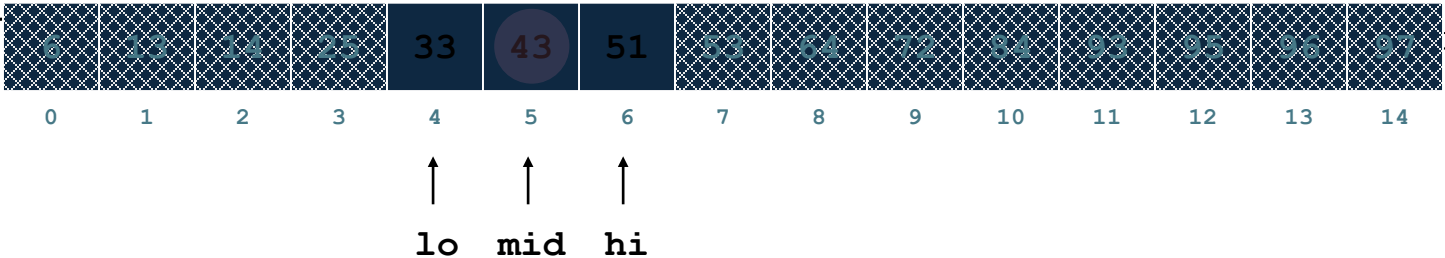
↑ ↑
lo hi

- Ex. Binary search for 33.

Binary Search

- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

- Invariant: `a[lo] ≤ value ≤ a[hi]`.



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
6	13	14	35	33	43	51	53	54	72	84	93	95	96	97

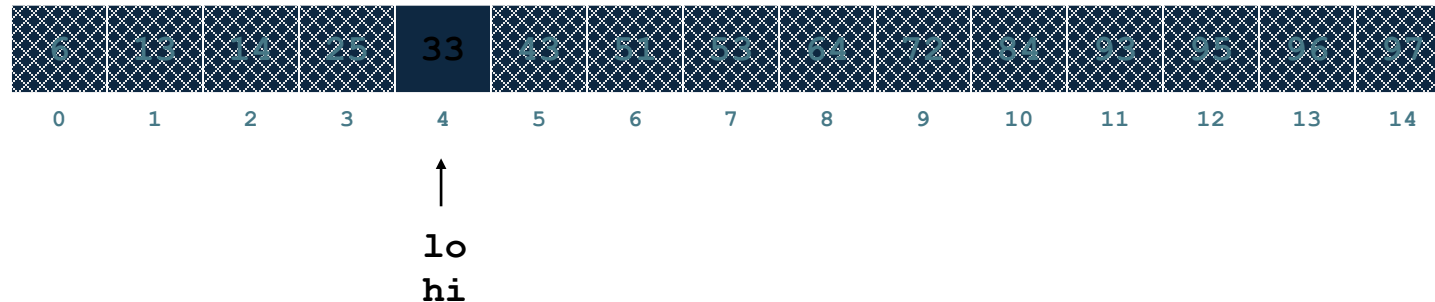
↑ ↑ ↑
lo mid hi

- Ex. Binary search for 33.

Binary Search

- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

- Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.

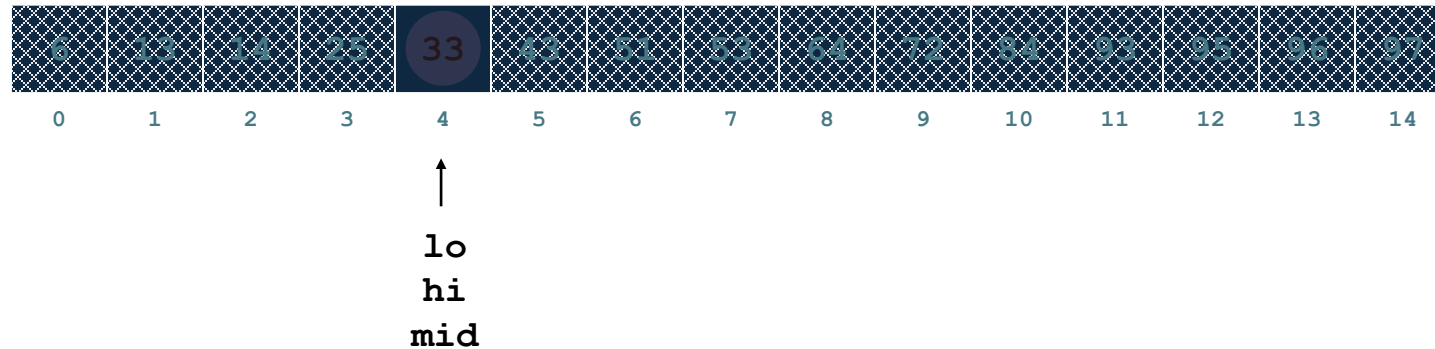


- Ex. Binary search for 33.

Binary Search

- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

- Invariant. Algorithm maintains $a[\text{lo}] \leq \text{value} \leq a[\text{hi}]$.

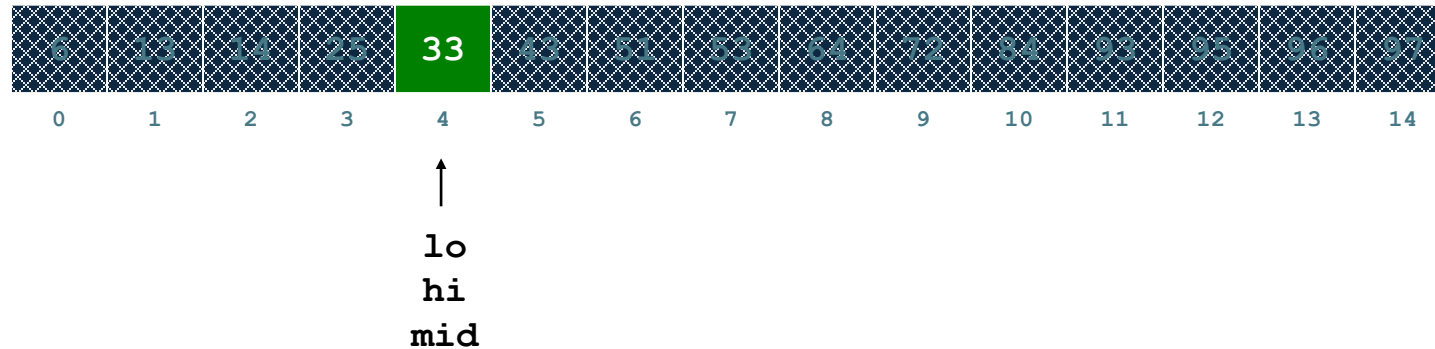


- Ex. Binary search for 33.

Binary Search

- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.

- Invariant. Algorithm maintains $a[lo] \leq value \leq a[hi]$.



- Ex. Binary search for 33.

10	20	50	100	200	500	2000
----	----	----	-----	-----	-----	------

Search

20

Example: Binary Search

- Searching for an element k in a sorted array A with n elements
- Idea:
 - Choose the middle element $A[n/2]$
 - **If $k == A[n/2]$,** we are done
 - **If $k < A[n/2]$,** search for k between $A[0]$ and $A[n/2 - 1]$
 - **If $k > A[n/2]$,** search for k between $A[n/2 + 1]$ and $A[n-1]$
 - Repeat until either k is found, or no more elements to search
- Requires a smaller number of comparisons than linear search in the worst case ($\log_2 n$ instead of n)


```

int main() {
    int A[100], n, k, i, mid, low, high;
    scanf("%d %d", &n, &k);
    for (i=0; i<n; ++i)
        scanf("%d", &A[i]);
    low = 0; high = n - 1; mid = (high + low)/2;
    while (high >= low) {

        if (A[mid] == k) {
            printf("%d is found\n", k);
            break;
        }
        if (k < A[mid])
            high = mid - 1;
        else
            low = mid + 1;
            mid = (high + low)/2;
    }

    If (high < low) printf("%d is not found\n", k);
}

```

Binary Search

Binary search operates by iteratively dividing the search range in half, progressively narrowing down the possibilities at each step.

We can denote the time taken for a dataset of size 'n' as $T(n)$, where $T(n/2)$ accounts for the time spent searching within one-half of the dataset, and $O(1)$ represents the constant time required for comparisons and calculations.

This can be expressed using the recurrence relation:

$$T(n) = T(n/2) + O(1)$$

Quick Sort

- This algorithm follows divide and conquer strategy
- Also known as partition exchange algorithm
- One of the fastest sorting algorithms
- Divide the list of numbers into sub-list until no more division possible.

How do we partition the array ?

- Choose pivot – a random element initially (generally 0th index element)

But choosing a right pivot is very important and affect the complexity of algorithm.

- Once pivot gets final position then,

<Pivot	Pivot	>Pivot
--------	-------	--------

Quick sort

Algorithm Quick_sort(A,low,high): This algorithm sort the elements in ascending order. A is linear array where low is equal to 0 and high is highest index i.e. n-1. A sub algorithm partition is used to split an array into two parts, and it returns final position of pivot element.

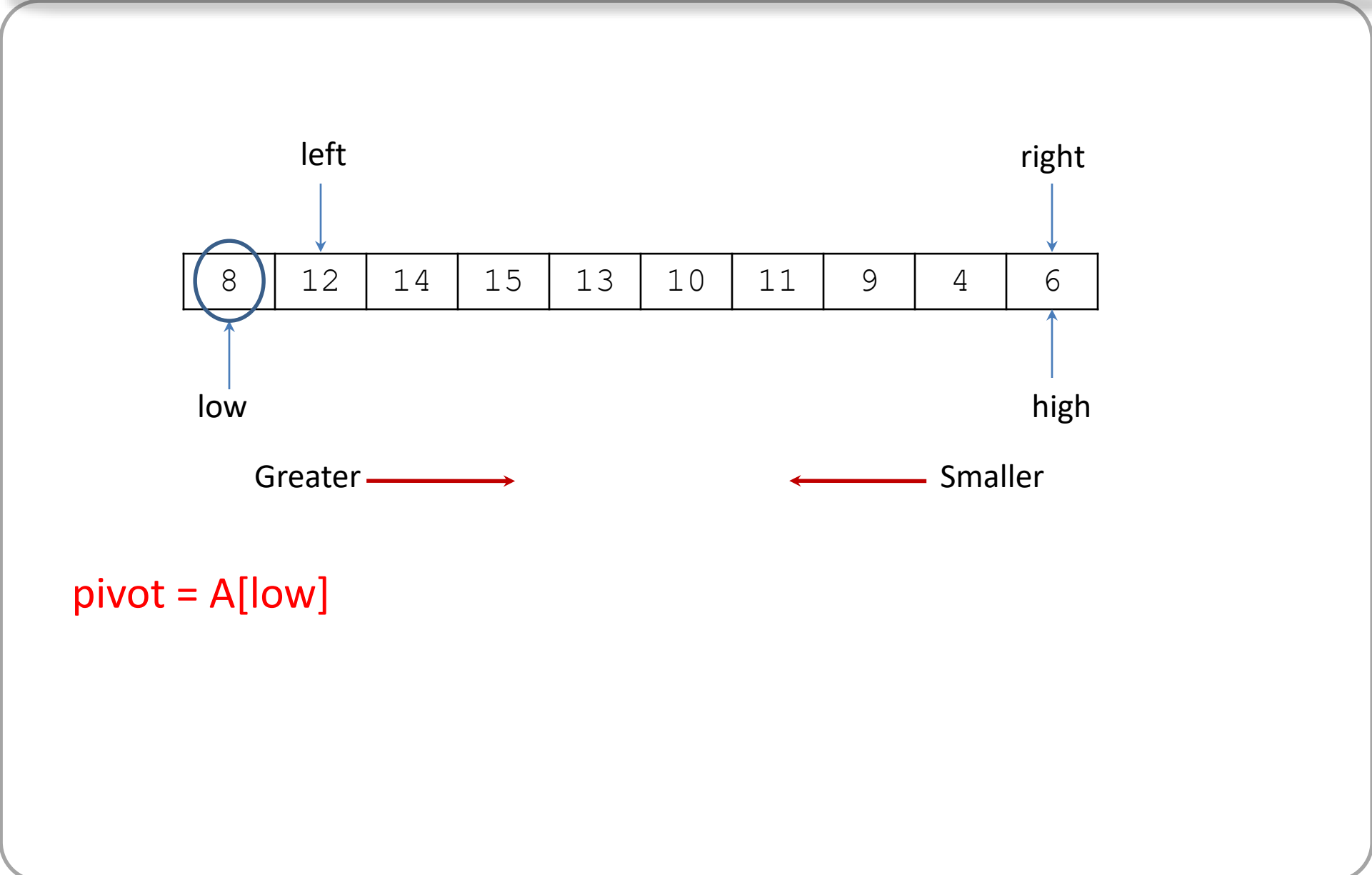
1. If $low < high$
2. $pivot = Partition(A, low, high)$
3. Quick_sort(A, low, pivot-1)
4. Quick_sort(A, pivot+1, high)

Quick Sort Partition

Partition(A, low, high) – This algorithm split the array into two parts and fixed the position of pivot element. Function swap is used to exchange the element in array.

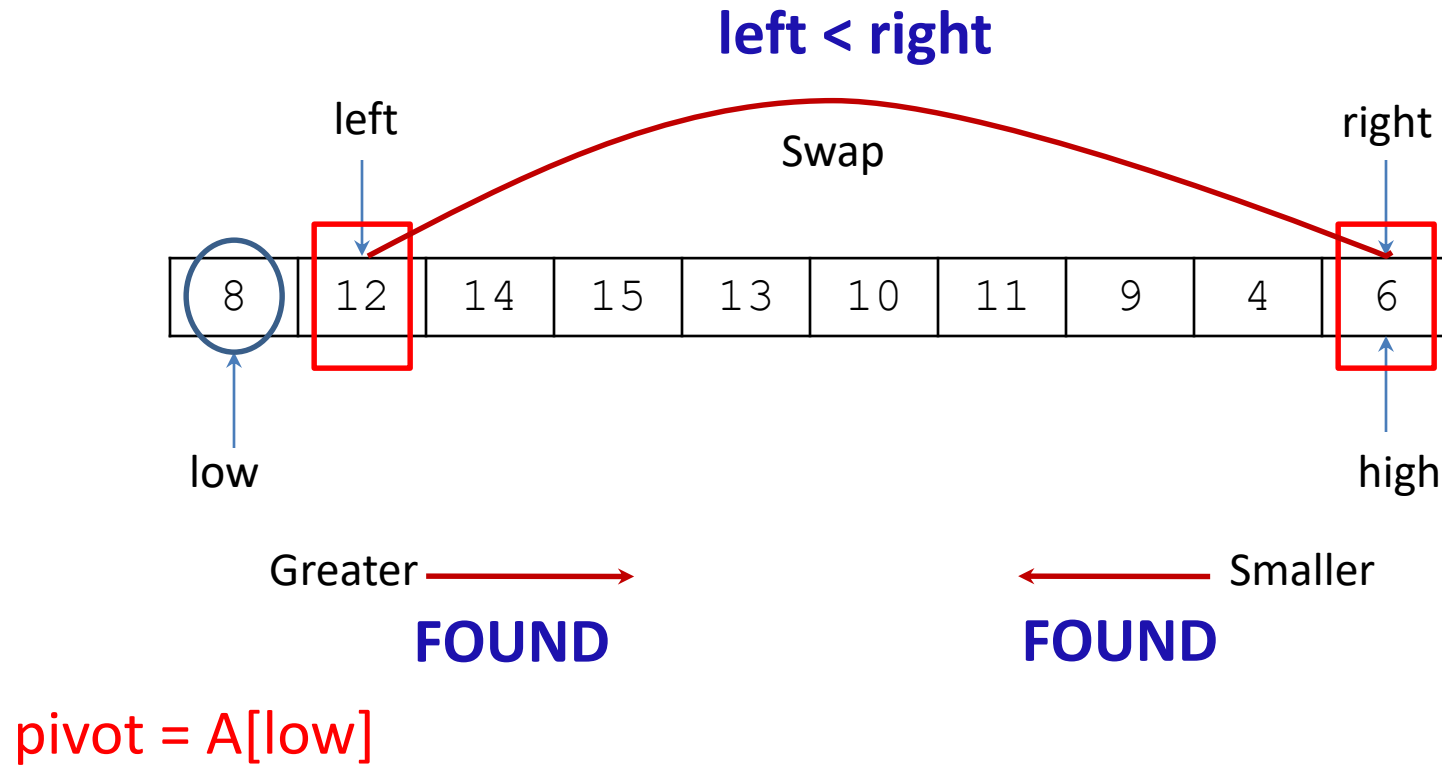
```
1.  pivot = A[low]
2.  left = low+1
3.  right = high
4.  While ( left <= right )
5.      While( A[left] < pivot )
6.          left=left+1
7.      While( A[right] > pivot )
8.          right=right-1
9.      If ( left < right )
10.         swap(A[left],A[right])
11. swap(A[right], pivot)
12. Return right
```

Quick Sort Illustration

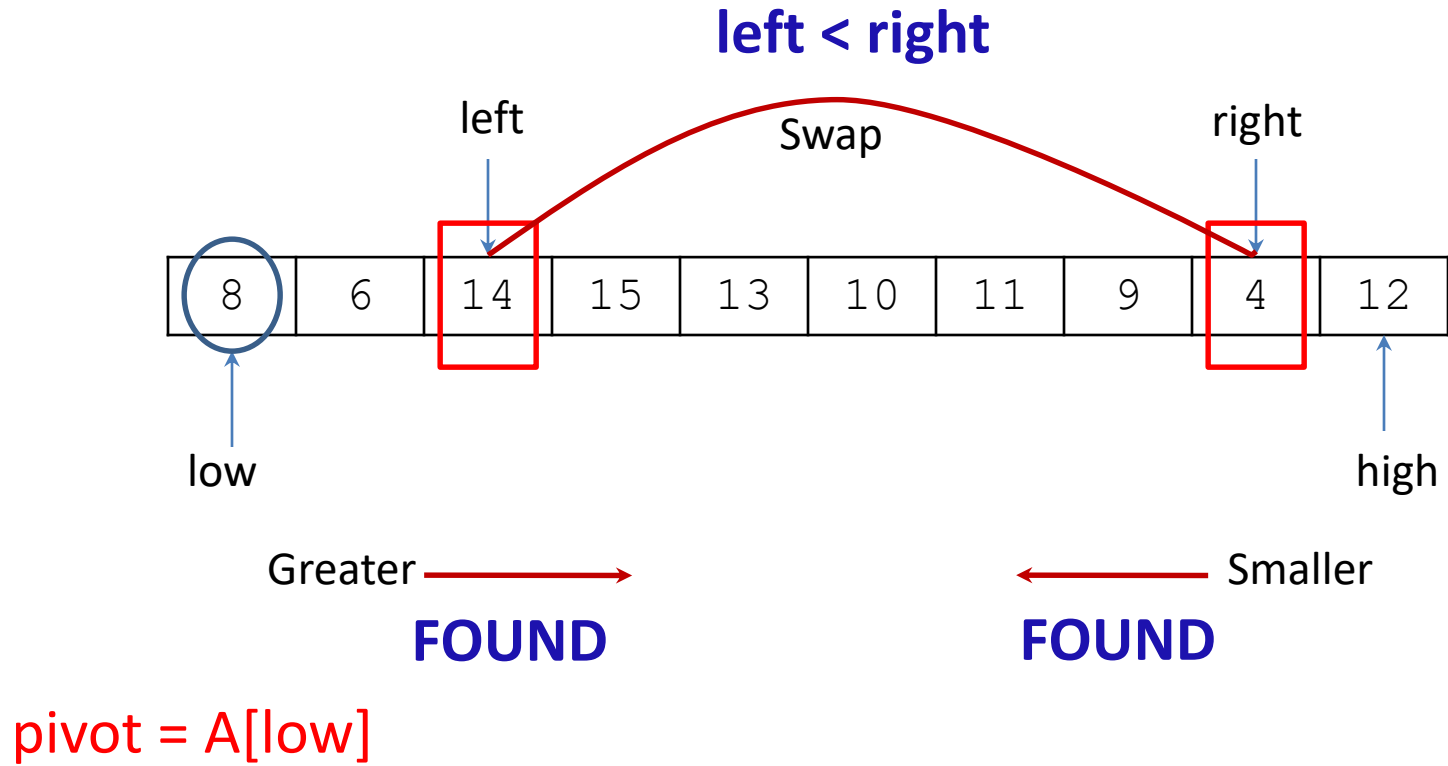


pivot = A[low]

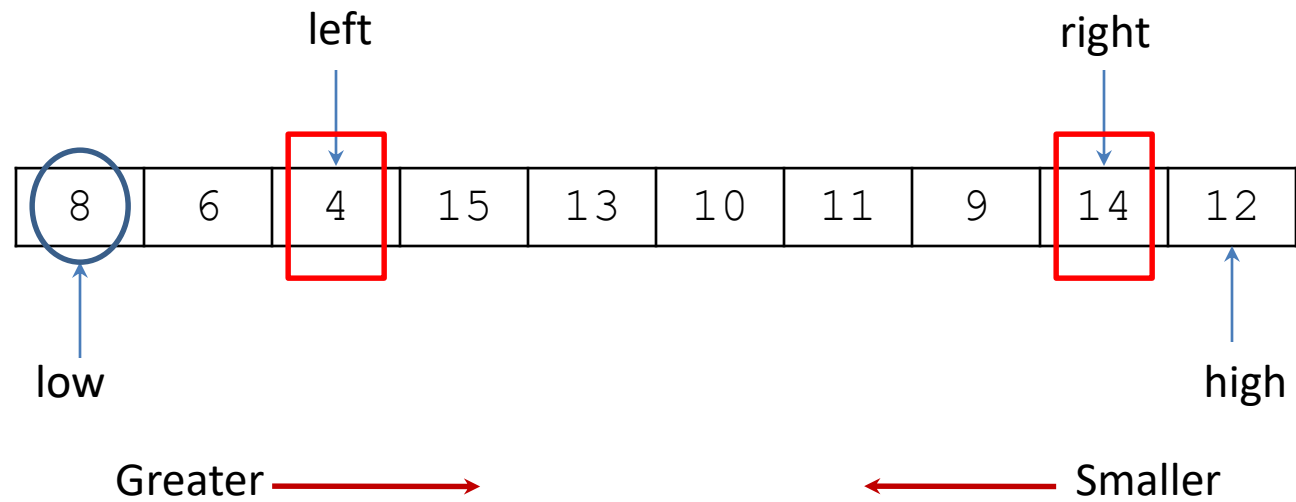
Quick Sort Illustration



Quick Sort Illustration

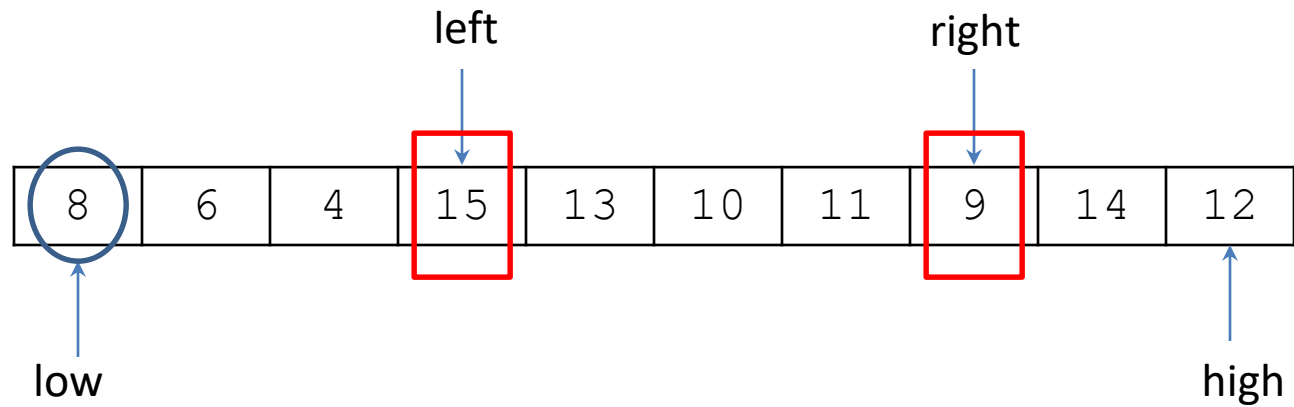


Quick Sort Illustration



$\text{pivot} = A[\text{low}]$

Quick Sort Illustration



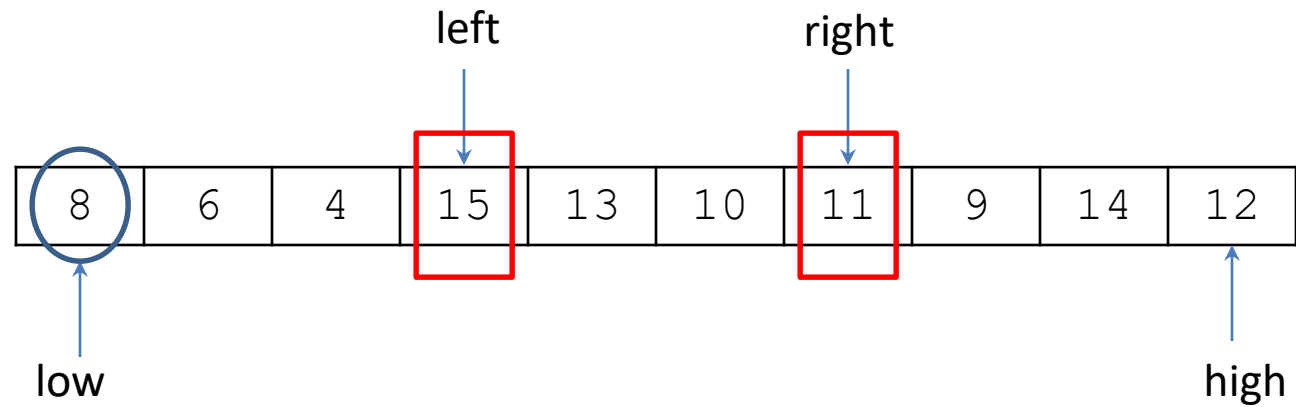
Greater $\longrightarrow$

$\longleftarrow$ Smaller

FOUND

pivot = A[low]

Quick Sort Illustration



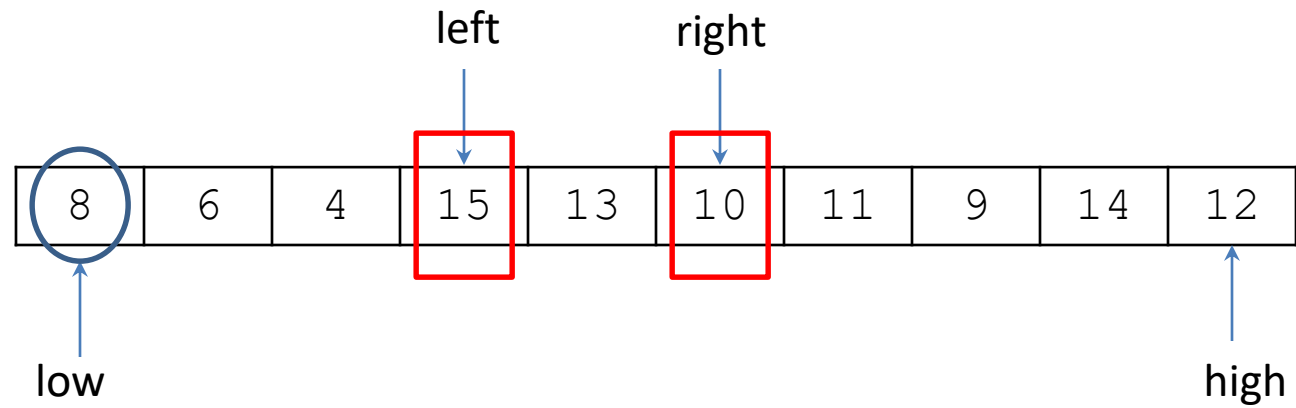
Greater $\longrightarrow$

$\longleftarrow$ Smaller

FOUND

pivot = A[low]

Quick Sort Illustration



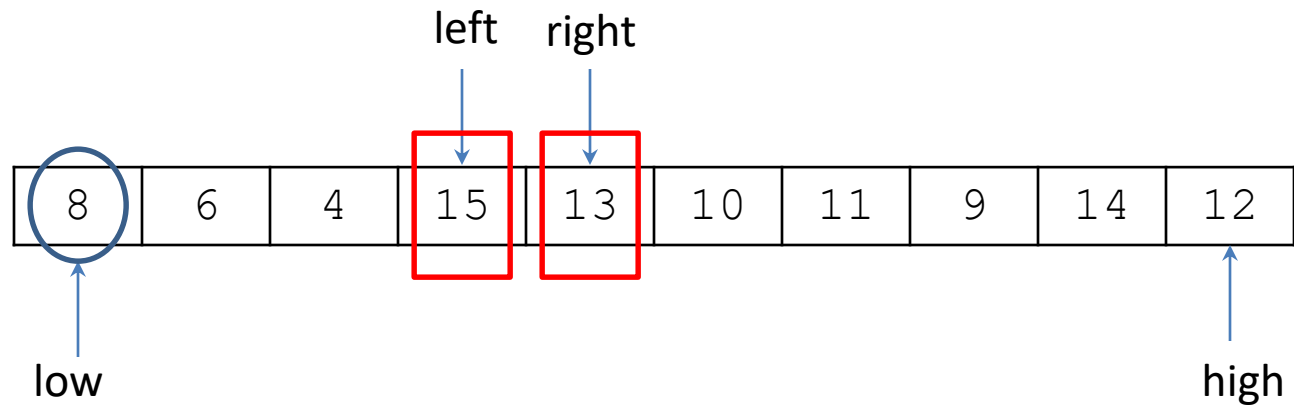
Greater $\longrightarrow$

$\longleftarrow$ Smaller

FOUND

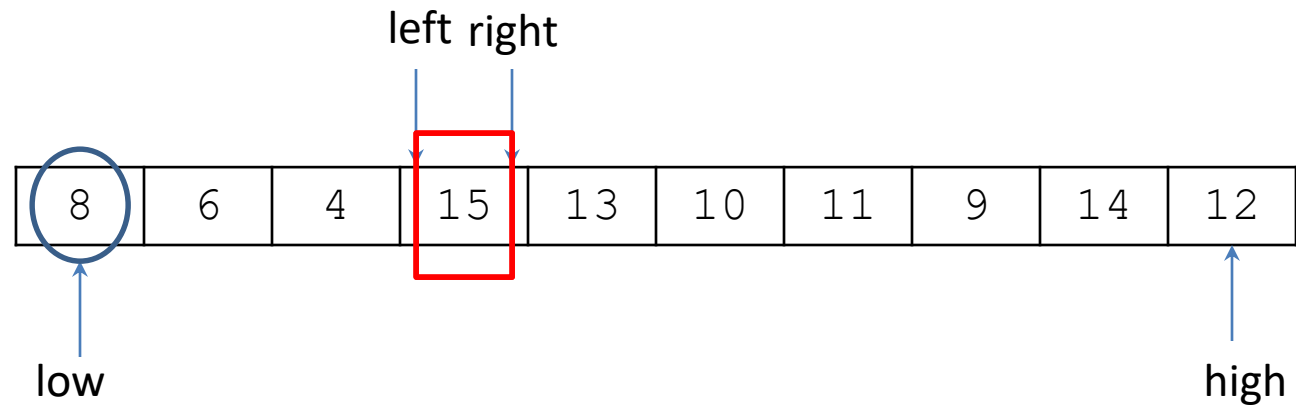
pivot = A[low]

Quick Sort Illustration



pivot = A[low]

Quick Sort Illustration



Greater →

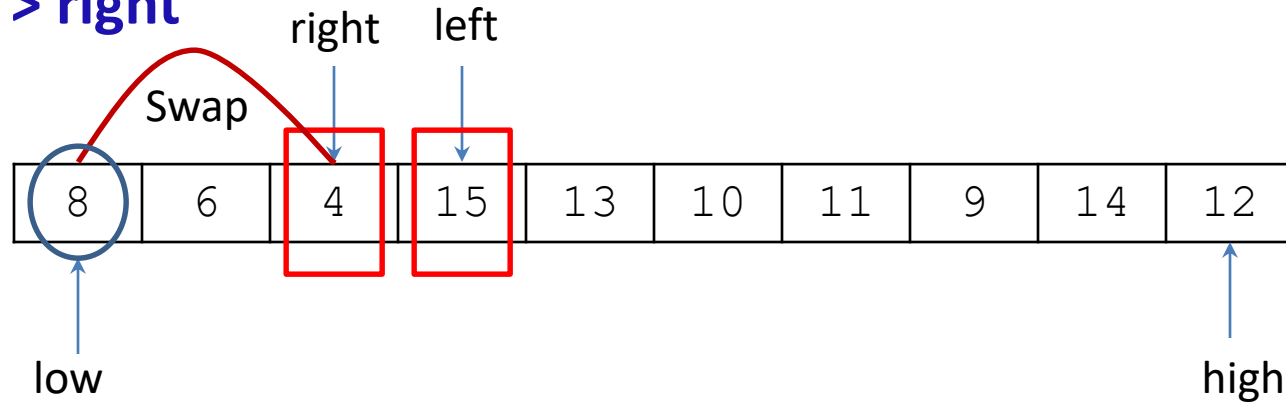
← Smaller

FOUND

pivot = A[low]

Quick Sort Illustration

left > right



Greater $\longrightarrow$

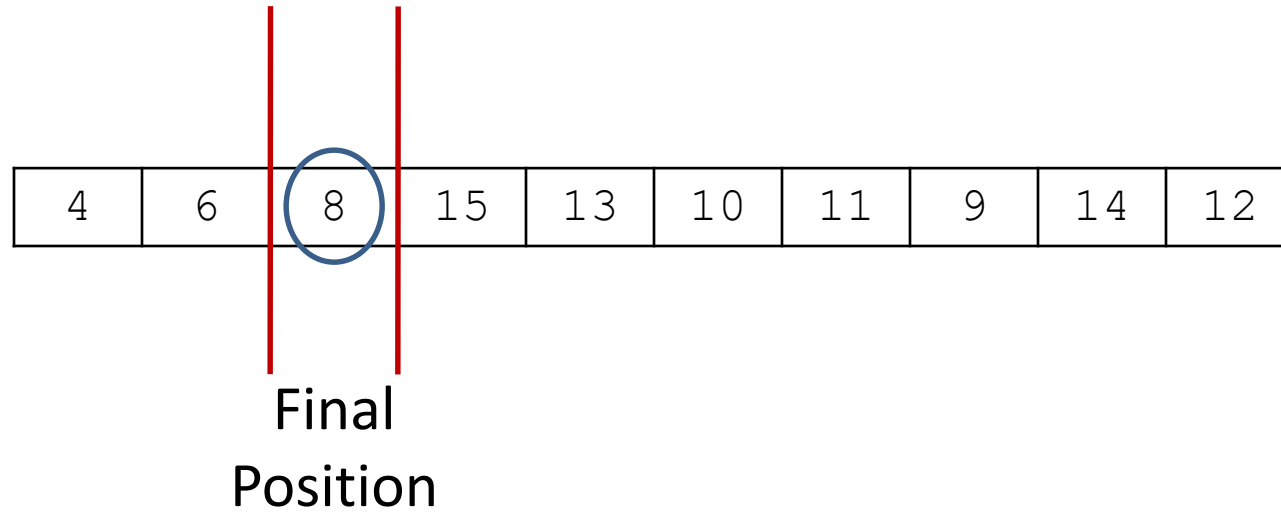
FOUND

$\longleftarrow$ Smaller

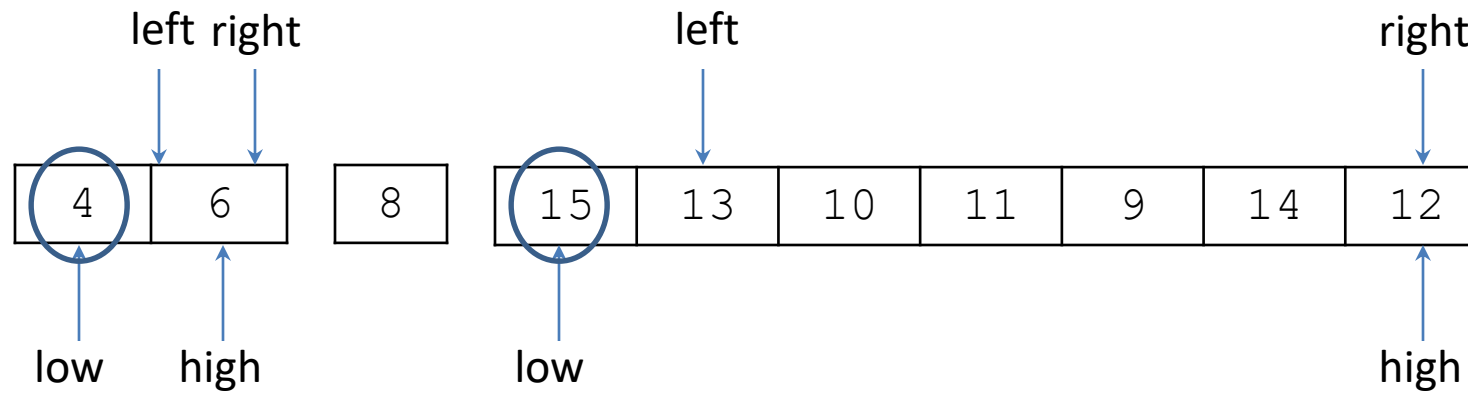
FOUND

pivot = A[low]

Quick Sort Illustration

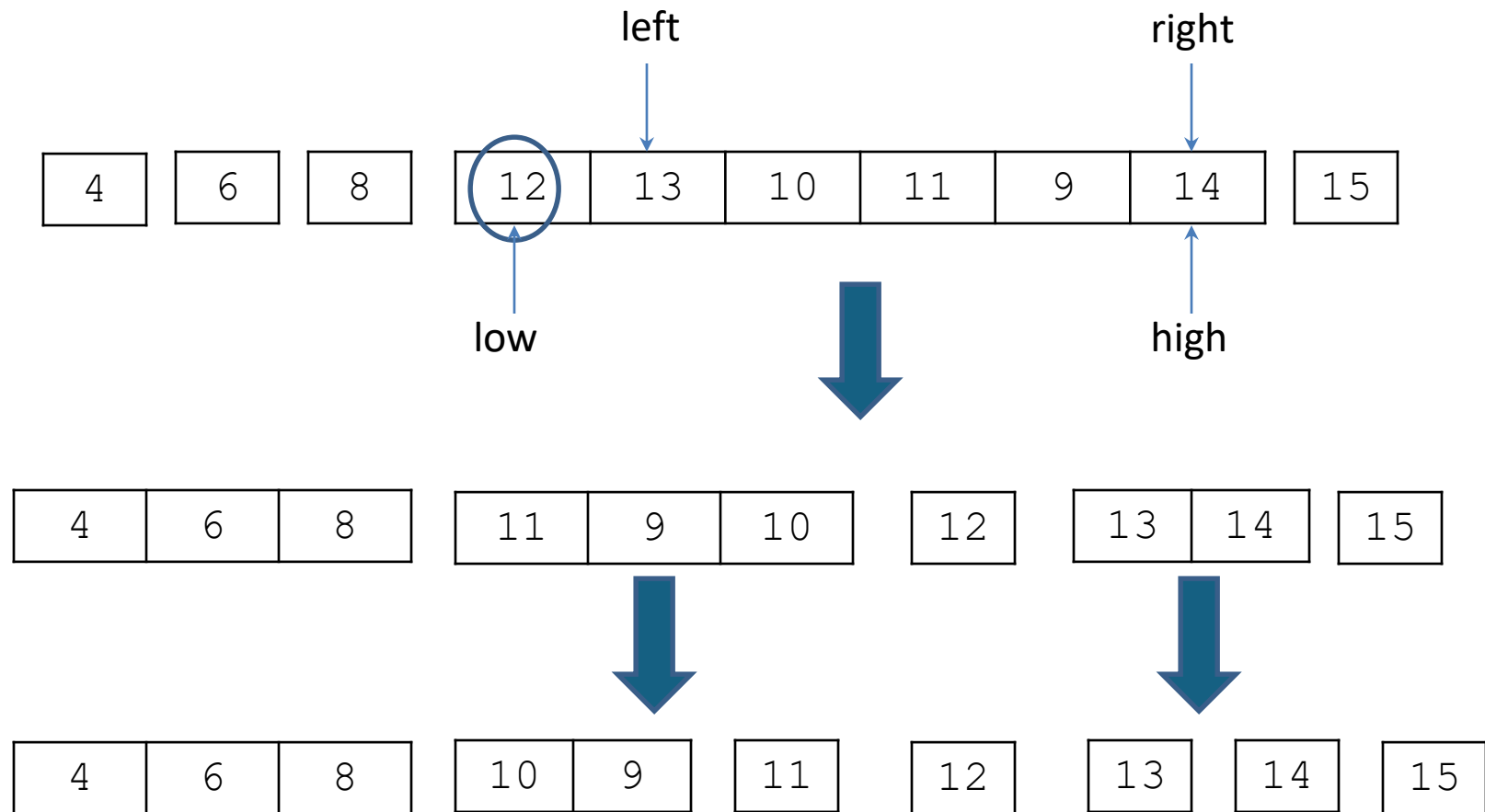


Quick Sort Illustration



pivot = A[low]

Quick Sort Illustration



Quick Sort Illustration

Final Output:

4	6	8	9	10	11	12	13	14	15
---	---	---	---	----	----	----	----	----	----

Quick Sort Illustration

Final Output:

4	6	8	9	10	11	12	13	14	15
---	---	---	---	----	----	----	----	----	----

Analysis of Quick Sort

Worst case:

When the list is already sorted.

Recurrence relation –

$$T(n) = \begin{cases} 1 & n=1 \text{ or } 0 \\ T(n-1) + n & n>1 \end{cases}$$

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + n-1$$

$$T(n-2) = T(n-3) + (n-2)$$

$$T(n-3) = T(n-4) + (n-3)$$

.....

$$T(3) = T(2) + 3$$

$$T(2) = T(1) + 2$$

Hence, $1 + 2 + 3 + \dots + (n-2) + (n-1) + n = n(n+1)/2$

Time Complexity = $O(n^2)$

Analysis of Quick Sort (Cont...)

Average case:

Recurrence relation –

$$T(n) = \begin{cases} 1 & n=1 \text{ or } 0 \\ 2T(n/2) + n-1 & n>1 \end{cases}$$

Time Complexity = $O(n \log_2 n)$

HINT: $2^k T(n/2^k) + kn - (2^k - 1)$

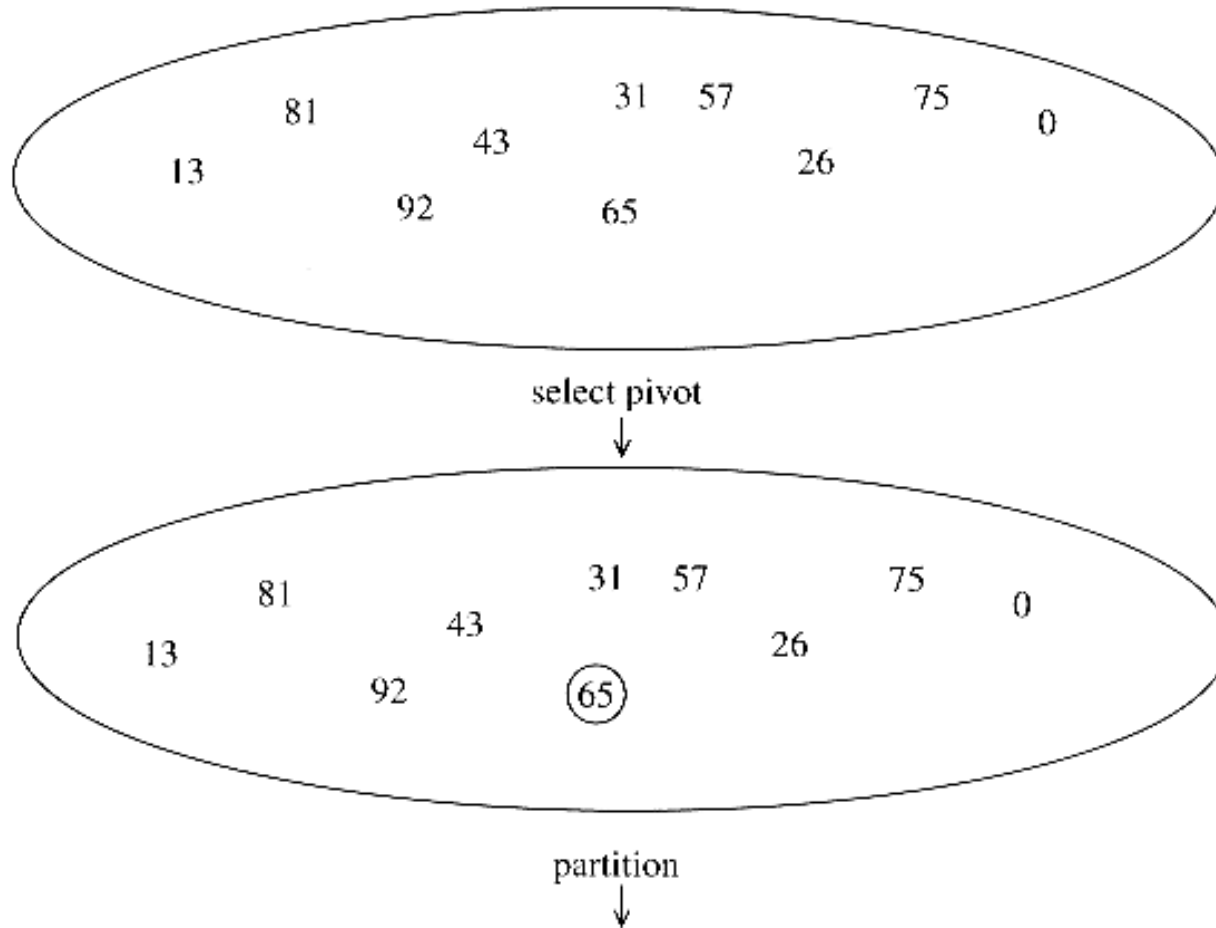
Quick Sort

- **Fastest** known sorting algorithm in practice
- Average case: $O(N \log N)$
- Worst case: $O(N^2)$
 - But the worst case can be made exponentially unlikely.
- Another divide-and-conquer recursive algorithm, like merge sort.

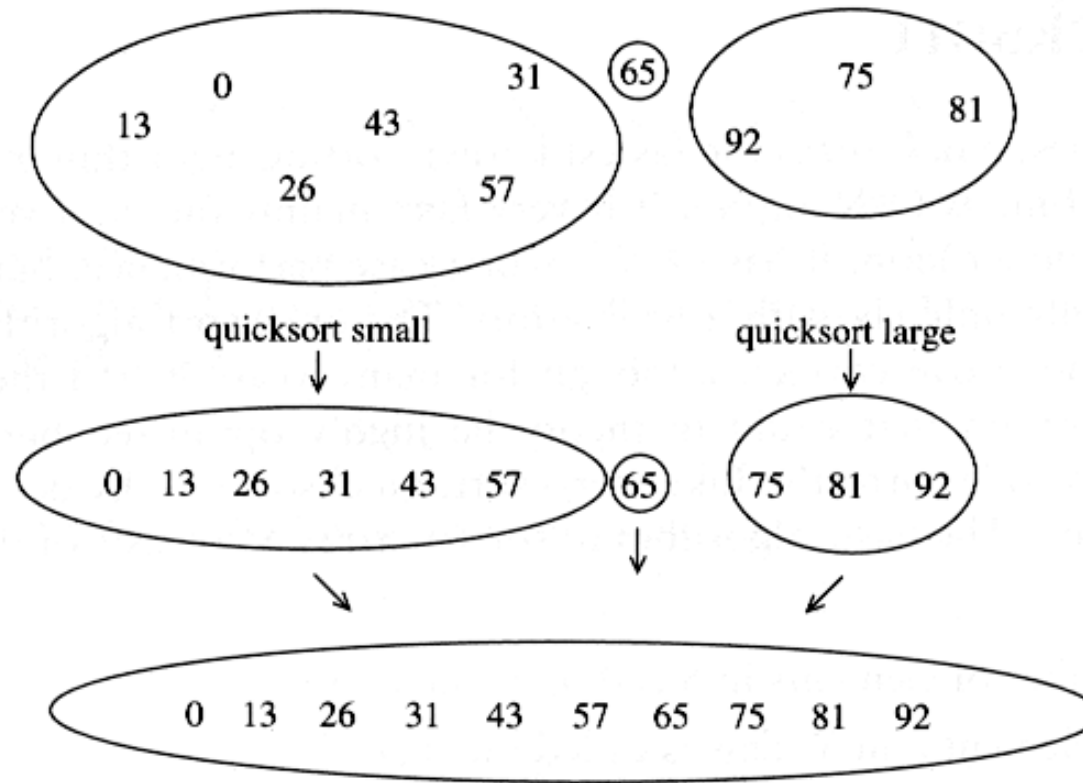
Quick Sort: Main Idea

1. If the number of elements in S is 0 or 1, then return (base case).
2. Pick any element v in S (called the pivot).
3. Partition the elements in S except v into two disjoint groups:
 1. $S_1 = \{x \in S - \{v\} \mid x \leq v\}$
 2. $S_2 = \{x \in S - \{v\} \mid x \geq v\}$
4. Return $\{\text{QuickSort}(S_1) + v + \text{QuickSort}(S_2)\}$

Quick Sort: Example



Example of Quick Sort...



Issues To Consider

- How to pick the pivot?
 - Many methods
- How to partition?
 - Several methods exist.
 - The one we consider is known to give good results and to be easy and efficient.
 - We discuss the partition strategy first.

Picking the Pivot

- There are several ways to pick a pivot.
- Objective: Choose a pivot so that we will get 2 partitions of (almost) equal size.

Picking the Pivot (2)

- Use the first element as pivot
 - if the input is random, ok.
 - if the input is presorted (or in reverse order)
 - all the elements go into S_2 (or S_1).
 - this happens consistently throughout the recursive calls.
 - results in $O(N^2)$ behavior (we analyze this case later).
- Choose the pivot randomly
 - generally safe,
 - but random number generation can be expensive and does not reduce the running time of the algorithm.

Picking the Pivot (3)

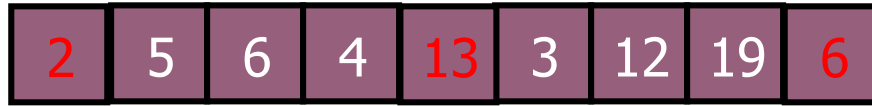
- Use the median of the array (ideal pivot)
 - The $\lceil N/2 \rceil$ *th* largest element
 - Partitioning always cuts the array into roughly half
 - An *optimal* quick sort ($O(N \log N)$)
 - However, hard to find the exact median
- Median-of-three partitioning
 - eliminates the bad case for sorted input.
 - reduces the number of comparisons by 14%.

Median of Three Method

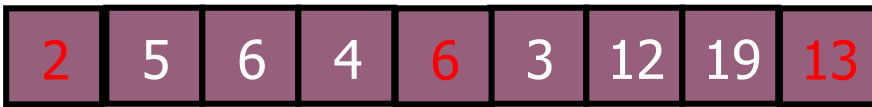
- Compare just three elements: the leftmost, rightmost and center
 - Swap these elements if necessary so that
 - $A[\text{left}] = \text{Smallest}$
 - $A[\text{right}] = \text{Largest}$
 - $A[\text{center}] = \text{Median of three}$
 - Pick $A[\text{center}]$ as the pivot.

```
int center = ( left + right ) / 2;  
if( a[ center ] < a[ left ] )  
    swap( a[ left ], a[ center ] );  
if( a[ right ] < a[ left ] )  
    swap( a[ left ], a[ right ] );  
if( a[ right ] < a[ center ] )  
    swap( a[ center ], a[ right ] );  
  
// Place pivot at position right - 1  
swap( a[ center ], a[ right - 1 ] );
```

Median of Three: Example



$A[\text{left}] = 2$, $A[\text{center}] = 13$,
 $A[\text{right}] = 6$

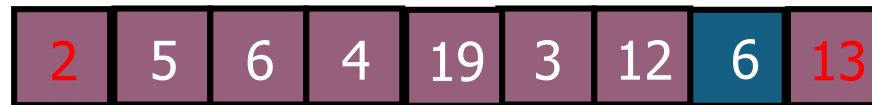


Swap $A[\text{center}]$ and $A[\text{right}]$



Choose $A[\text{center}]$ as **pivot**

↑
pivot



Swap pivot and $A[\text{right} - 1]$

↑
pivot

We only need to partition $A[\text{left} + 1, \dots, \text{right} - 2]$.